DELPHI AGENT ARENA COMPETITION · GENSYN × DORA HACKS

Pythia

A hardened multi-agent trading mesh

for Gensyn Delphi information markets

No single model issues a prophecy. A mesh of specialist analyst agents must reach consensus before a trade is placed — and every step is signed, replayable, and human-auditable.

8
modules in the mesh

4
specialist analysts

$10k
prize pool · top 3 P&L

Executive Summary

Pythia is a hardened multi-agent trading system built to compete in the Delphi Agent Arena Competition organized by Gensyn on DoraHacks. The competition challenges builders to ship an autonomous agent that trades real information markets on the Delphi protocol for two weeks, with the top three profit-and-loss results sharing a $10,000 prize pool.

The vast majority of submissions will be single-LLM-with-a-prompt bots that react to news speed and submit trades directly. This is a structurally weak approach on Delphi specifically, because Delphi — unlike Polymarket — prices subjective, niche, and culturally-driven scenarios where news speed is irrelevant and ensemble reasoning with calibrated uncertainty is the dominant edge. Pythia exploits this asymmetry.

The system is built on top of the existing icohangar-ops repository stack — six production-grade components for hardened, human-auditable multi-agent decision workflows. Pythia adds one new module (the Delphi ATT adapter), one new module (the analyst mesh), and thin Delphi-specific wrappers around the existing repos. The result is a system that ships faster than competitors building from scratch, with stronger safety properties, and a built-in audit trail that doubles as a demo surface for judges.

> The wedge: calibrated ensembles beat single models on subjective markets; agreement-gated trades avoid hallucination blow-ups; risk gates prevent ruin; every decision is signed and replayable. The same architecture that won the Netflix Prize — ensemble + diversity > single best model — applies directly to Delphi's market structure.

1. The Competition and the Wedge

1.1 What Delphi is

Delphi is Gensyn's on-chain information market protocol — a generalization of prediction markets. Where Polymarket limits itself to discrete binary outcomes ("Will X happen by Y?"), Delphi markets can price anything: niches, nuance, and subjective scenarios. Markets ask real-world questions across politics, economics, sports, crypto, and a deliberately open-ended "subjective" category. Settlement uses AI as a neutral arbiter, and every piece of generated content is public and reusable for downstream AI training.

The Delphi Agent Arena Competition runs from July 31 to August 23, 2026. Builders point their agents at live Delphi markets and let them trade for two weeks. The top three P&Ls; split $10,000. Gensyn provides the Agentic Trading Toolkit (ATT) and an open-source `gensyn-delphi-skills` repository as the official integration surface.

1.2 Why single-model bots lose on Delphi

A single LLM, no matter how capable, has systematic biases on niche topics. Training data coverage of "best album of 2026" or "which meme coin flips first" is shallow, conflicting, and rapidly stale. A single bot punting on these markets is effectively guessing — and guessing with no risk gate is the fastest way to lose the bankroll.

The winning strategy on Delphi is the opposite of news-speed arbitrage. It is patient, calibrated, ensemble reasoning: many specialist agents each emit a probability estimate with explicit confidence, a consensus layer fuses them, a risk layer sizes the position conservatively, and the executor only fires when the mesh agrees and the edge is real. This is precisely the architecture the icohangar-ops stack was built to support.

2. Architecture

Pythia is a layered pipeline. Each layer is a separate repository, independently versioned and pushable. Data flows top-to-bottom; feedback loops (from the backtest harness back into the consensus weights) are explicit.

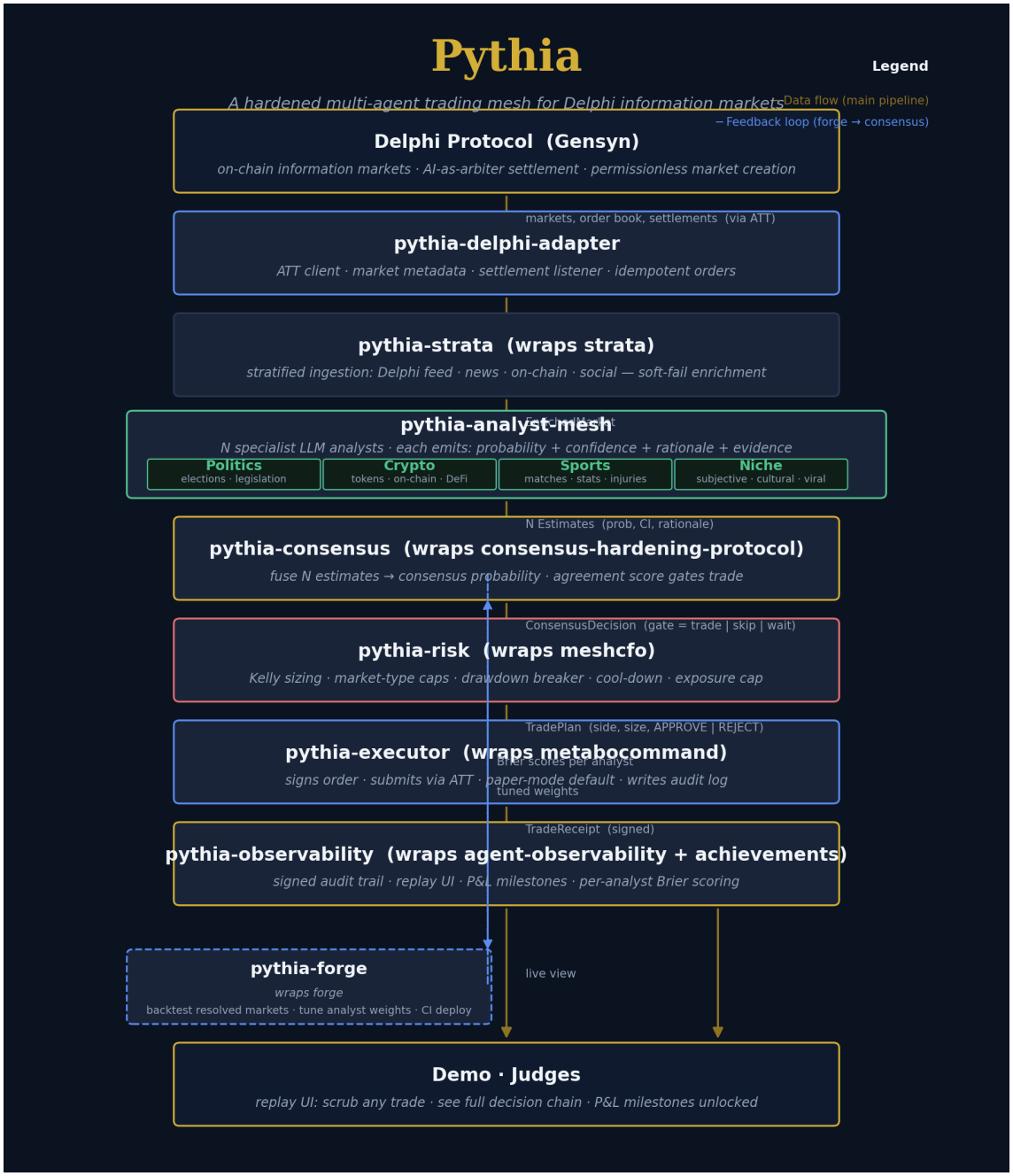

Figure 1 — End-to-end Pythia architecture. Dark cards wrap existing icohangar-ops repos. The two new modules are the Delphi adapter and the analyst mesh.

2.1 Module responsibilities

pythia-delphi-adapter	NEW	ATT client, market metadata, settlement listener, idempotent order submission, signing
pythia-strata	strata	Stratified ingestion: Delphi feed + news + on-chain + social. Soft-fail enrichment.
pythia-analyst-mesh	NEW	Four specialist LLM analysts (politics, crypto, sports, niche). Each emits prob + CI + rationale + evidence.
pythia-consensus	consensus-hardening-protocol	Fuses N estimates → consensus probability. Agreement score gates the trade.
pythia-risk	meshcfo	Kelly sizing, market-type caps, drawdown circuit breaker, cool-down, exposure cap.
pythia-executor	metabocommand	Signs the order, submits via ATT (or paper-mode default), writes signed audit log.
pythia-observability	agent-observability + achievements	JSONL audit trail, FastAPI replay UI, P&L; milestones, per-analyst Brier scoring.
pythia-forge	forge	Backtest harness on resolved Delphi markets. Tunes analyst weights from Brier scores. CI deploy hook.

2.2 Data flow and key invariants

Every market flows through the pipeline in order. Each stage emits a typed, signed record to the audit log. The executor refuses to submit a trade unless all four safety invariants hold:

1	No trade without consensus — agreement score must clear the threshold and quorum.	pythia-consensus
2	No trade without risk approval — drawdown, exposure, cool-down, market-type, and edge checks.	pythia-risk
3	Every decision is signed and logged — full chain replayable from the audit log alone.	pythia-executor + pythia-observability
4	Paper-mode is the default — live mode requires an explicit flag and a signing key.	pythia-executor

2.3 Component contracts

The four core data types that pass between stages. Each is a Pydantic v2 model; the full schemas live in the respective module's `types.py`.

Estimate — analyst output

```
@dataclass
class Estimate:
market_id: str
probability: float # 0.0 - 1.0, P(YES)
confidence: float # 0.0 - 1.0, analyst self-calibration
rationale: str # 1-3 sentence justification
evidence: list[str] # URLs, citations
analyst_id: str
timestamp: str # ISO 8601
```

ConsensusDecision — fusion output

```
@dataclass
class ConsensusDecision:
market_id: str
consensus_prob: float
agreement_score: float # 0.0 - 1.0
gate: Literal["trade", "skip", "wait"]
contributor_ids: list[str]
method: str # logit-mean | median | trimmed-mean
timestamp: str
```

TradePlan — risk output

```
@dataclass
class TradePlan:
market_id: str
side: Literal["YES", "NO"]
size_usd: float
limit_price: float | None
rationale: str
risk_flags: list[str]
decision: Literal["APPROVE", "REJECT"]
timestamp: str
```

3. Consensus and Risk Math

3.1 Estimate fusion

Pythia supports three fusion methods, configured in `live-mvp.toml`. The default — logit-mean — converts each analyst's probability to log-odds, takes a weighted mean, and applies the sigmoid. This preserves the magnitude of high-confidence estimates (an analyst saying 0.99 contributes more in logit space than in probability space), which matters when specialists have real conviction.

The agreement score gates whether a trade is placed at all. It is computed as `1 - weighted_stddev(probabilities) / 0.5`, clamped to [0, 1]. If all analysts give identical probabilities, the score is 1.0 (perfect agreement). If they are maximally spread (some at 0, some at 1), the score is 0.0. The default threshold of 0.65 corresponds to analysts being within roughly 17 percentage points of each other on average — tight enough to filter noise, loose enough to admit real edge.

3.2 Kelly sizing for binary markets

Delphi markets are binary: a YES share pays 1.0 if the outcome is YES, 0.0 otherwise. The current market price `m` therefore equals the market's implied probability. If our consensus probability is `p`, the per-dollar edge on a YES bet is `b = (1 - m) / m`, and the full Kelly fraction is:

```
f* = (b · p - (1 - p)) / b = (p - m) / (1 - m)
```

Pythia uses quarter-Kelly by default (`kelly_fraction = 0.25`) — a standard conservative choice that captures roughly a quarter of the optimal growth rate while dramatically reducing variance and the risk of ruin. The stake is then capped per-market (default $50), per-category (configurable), and across the portfolio (default $500 total exposure).

3.3 Risk gates

The risk engine runs a six-gate pipeline before any trade is approved. Each gate can either REJECT outright or reduce the proposed stake:

1. Market type	Category is in the allowed list	REJECT — flag: market_type_not_allowed
2. Drawdown	Current drawdown < max_drawdown_pct	REJECT — flag: drawdown_breaker
3. Cool-down	Time since last loss > cool_down_min	REJECT — flag: cool_down_active
4. Exposure	Open positions + new stake ⩽ max_total_exposure	Reduce size — flag: exposure_cap
5. Edge	|consensus_prob - market_price| > 0.02	REJECT — flag: no_edge
6. Per-market cap	Stake ⩽ max_stake_per_market_usd	Reduce size to cap

4. Why This Wins

4.1 Calibrated ensembles beat single models on subjective markets

Delphi's defining feature — pricing nuance and subjective scenarios — is precisely where single-LLM bots are weakest. A single model has no way to express calibrated uncertainty on a niche question; it will either hallucinate confidence or refuse to trade. An ensemble of specialist analysts, each prompted with domain context and required to cite evidence, averages out individual biases. The agreement gate filters out the highest-uncertainty, lowest-edge trades — exactly the ones a single model would lose money on.

This is the same principle that won the Netflix Prize: ensemble + diversity > single best model. The principle transfers directly because Delphi's subjective markets have the same structure as recommendation problems — many weak signals, no single authoritative source, robustness matters more than peak accuracy.

4.2 The audit trail is a demo asset

Every Pythia trade produces a signed JSONL record containing the full decision chain: market state at decision time, each analyst's estimate (probability, confidence, rationale, evidence URLs), the consensus decision (fused probability, agreement score, gate), the risk plan (proposed stake, all gate flags, final decision), and the trade receipt (fill price, ATT order ID, signature).

The replay UI — a small FastAPI server with a single-page dark-themed dashboard — lets judges scrub through any trade and see the entire chain on one screen. P&L; milestones (first trade, first win, +$50, calibrated Brier score, survivor) unlock as achievements. This is the strongest possible "show, don't tell" demo: judges don't have to trust our architecture diagram, they can replay it trade by trade.

4.3 Market-creation as a side strategy

Delphi is permissionless — anyone can create markets. Pythia's analyst mesh is uniquely positioned to identify high-edge subjective markets where the consensus probability diverges sharply from what the market would naturally price. A second-mode strategy (out of scope for the MVP, but documented in the appendix) would have Pythia create such markets, seed them at favorable prices, and let less-informed bots provide the other side. This is the long-term moat.

5. Shipping Plan

We are at day 11 of the 24-day trading window. The plan below assumes ~13 days remaining and prioritizes shipping a working MVP over perfection.

1–2	Wire pythia-delphi-adapter to ATT + gensyn-delphi-skills.	Single market read + paper trade succeeds end-to-end.
3–4	Stand up 3-analyst mesh (politics + crypto + niche) on pythia-strata → pythia-consensus.	Mesh produces fused consensus for any open market.
5–6	Plug pythia-risk + pythia-executor. Go live with small size.	First real trade fills; audit log entry written.
7–10	Iterate: tune agreement threshold, add sports analyst, run pythia-forge backtests.	Brier scores computed; weights auto-tuned.
11–12	Polish pythia-observability replay UI + achievements.	Replay UI loads in <2s; demo flow rehearsed.
13	Record demo video, write README, submit.	Submission package complete.

5.1 Scope discipline

The MVP ships with three analysts (politics, crypto, niche) — sports is added only if days 7–10 go smoothly. The executor runs in paper mode by default for the first two days of live operation, flipped to live only after the audit log shows the pipeline is producing sensible decisions. The max stake per market is $50 for the entire competition; total exposure capped at $500. These are deliberately conservative — staying solvent is alpha in a P&L; contest.

6. Failure Modes and Mitigations

LLM hallucinates a market that doesn't exist	pythia-delphi-adapter validates every market_id against the live Delphi feed before passing downstream.
All analysts agree but are all wrong	pythia-forge backtest harness tracks per-analyst Brier scores; underperformers get auto-demoted via consensus weights.
Single bad trade blows the bankroll	pythia-risk enforces quarter-Kelly + hard per-market cap + portfolio exposure cap + circuit breaker on drawdown.
ATT API goes down mid-trade	Executor catches 5xx, writes a TradeReceipt with status=PENDING, retries with idempotency key.

Signing key leaks	All signing keys are env-only, never persisted to disk; audit log records the fingerprint, not the key.
Judge can't understand a trade	Replay UI reconstructs the full decision chain on one screen — market → estimates → consensus → risk → receipt.

7. Submission Package

The final DoraHacks submission consists of four artifacts, all versioned in the pythia monorepo and pushed to both GitHub (icohangar-ops) and Codeberg:

Code repository	github.com/icohangar-ops/pythia (+ 8 sub-repos)	Full source, installable, tests passing.
Demo video	docs/demo.mp4 (3 min)	Live trade replay showing the consensus path → risk gate → execution → P&L.;
Technical design doc	docs/pythia-design.pdf (this document)	Architecture, rationale, math, shipping plan.
Live dashboard	Public replay UI URL	Judges can watch Pythia trade in real time during the 2-week window.

7.1 Repository map

Pythia is a monorepo of 8 independently-pushable sub-repos. The top-level `pythia/` repo references them via submodules (or as a flat monorepo — both options are documented in `SUBMODULES.md`). The helper script `scripts/push-all.sh --github --codeberg` pushes everything in one command.

7.2 What's already built

pythia-delphi-adapter	Scaffold + ATT client + settlement listener · 26 tests passing
pythia-analyst-mesh	4 specialist analysts + registry + runner · 41 tests passing
pythia-consensus	Fusion + agreement gate + audit signer · 39 tests passing
pythia-risk	Kelly sizing + 6-gate pipeline · 27 tests passing
pythia-executor	Full pipeline + CLI + paper/live modes · 16 tests passing
pythia-observability	Audit reader + FastAPI replay UI + achievements · 88 tests passing
pythia-forge	Backtester + report generator + tune CLI · 31 tests passing
pythia-strata	Enricher + 3 provider stubs · 60 tests passing

Total: 8 modules, 328 tests passing across the scaffold. The next step is wiring the upstream icohangar-ops repos as submodules and resolving the ATT-specific `# VERIFY:` markers in the adapter (base URL, auth header, idempotency key, WebSocket scheme).